# The Constant Bandwidth Server: Bandwidth Reservation and Temporal Isolation for Aperiodic Tasks under EDF Scheduling

Soaib Ferdous
*BEng in Electronic Engineering*
*Hochschule Hamm-Lippstadt*
Student ID: 1232368, Group B,
soaib.ferdous@stud.hshl.de

***Abstract*—This paper presents the Constant Bandwidth Server (CBS), a dynamic-priority scheduling mechanism designed to serve aperiodic tasks alongside periodic hard real-time tasks without violating schedulability guarantees. Unlike earlier server approaches such as the Polling Server and Deferrable Server, the CBS formally bounds its CPU demand to an assigned bandwidth $U_s = Q_s/T_s$, even in the presence of task overruns. The paper covers the formal CBS definition, the three-rule scheduling algorithm, the isolation theorem and feasibility lemma, a worked scheduling example, a comparison with related aperiodic server approaches, a critical evaluation of CBS strengths and limitations, and application examples in embedded systems. The CBS provides a practical and formally correct mechanism for mixed workload scheduling, at the cost of requiring an EDF-capable runtime environment.**

***Index Terms*—real-time scheduling, aperiodic tasks, Constant Bandwidth Server, EDF, temporal isolation, bandwidth reservation, embedded systems**

## I. Introduction

Real-time embedded systems must often handle two co-existing task classes: periodic tasks with known execution times and strict hard deadlines, and aperiodic tasks that arrive unpredictably and carry soft or firm deadlines. The central scheduling challenge is to provide responsive service to aperiodic tasks without compromising the timing guarantees of the periodic workload.

Naive approaches fail at this problem in characteristic ways. Executing aperiodic tasks in the background minimizes interference but leads to unacceptably poor aperiodic response times. Treating aperiodic tasks as foreground tasks can cause periodic deadline misses. Server-based approaches reserve a dedicated execution budget for aperiodic work, but early designs such as the Polling Server [1] waste budget when no aperiodic job is waiting, and the Deferrable Server [1] can violate fixed-priority schedulability bounds through its budget-preservation mechanism.

The Constant Bandwidth Server (CBS), introduced by Abeni and Buttazzo [2] and extended in [3], resolves these issues by operating under Earliest Deadline First (EDF) scheduling and formally guaranteeing that the server's CPU contribution is bounded by $U_s$, independently of aperiodic task behavior. This paper analyzes the CBS in detail, drawing primarily on Buttazzo [1], Chapter 6, pp. 189–203.

## II. Background: The Aperiodic Server Problem

A set of $n$ periodic hard real-time tasks $\tau_i = (C_i, T_i)$ with implicit deadlines ($D_i = T_i$) is schedulable by EDF if and only if:

$$\sum_{i=1}^{n} \frac{C_i}{T_i} \leq 1 \tag{1}$$

When aperiodic tasks are added to the system, the goal is to serve them without violating Eq. (1). Table I summarizes how existing server approaches address this problem.

TABLE I
Aperiodic Server Comparison

| Server | Sched. | Guarantee | Key limitation |
|---|---|---|---|
| PS | RM | Yes | Budget wasted if idle |
| DS | RM | No | Can violate RM bound |
| SS | RM | Yes | Complex replenishment |
| TBS | EDF | Yes | No isolation under overrun |
| **CBS** | **EDF** | **Yes** | **Requires EDF runtime** |

The Deferrable Server is the most directly relevant predecessor. It improves aperiodic response time by preserving unused budget within a server period, but this preservation can cause back-to-back execution of up to $2Q_s$ in a short interval, effectively violating the schedulability assumptions of the periodic task set [1]. The CBS eliminates this problem through its deadline assignment rules.

The Total Bandwidth Server (TBS) operates under EDF and assigns each arriving aperiodic job $k$ the deadline $d_k = \max(r_k, d_{k-1}) + C_k/U_s$ [1]. TBS is simpler than CBS but uses the *actual* computation time $C_k$, which is only known at job completion. This means TBS cannot provide isolation against overrunning tasks — a job that executes longer than expected pushes all subsequent deadlines further out. The CBS resolves this through an explicit budget variable [2].

## III. CBS Definition

Following Buttazzo [1], pp. 189–190, a CBS is characterized by:

- $Q_s$: the maximum server budget (execution units per period)
- $T_s$: the server period
- $U_s = Q_s/T_s$: the server bandwidth
- $c_s$: the current remaining budget, $0 < c_s \leq Q_s$
- $d_{s,k}$: the current server deadline, initialized as $d_{s,0} = 0$

Each aperiodic job served by the CBS is assigned the current server deadline $d_{s,k}$ as its scheduling deadline and is inserted into the EDF ready queue. Whenever a job executes, $c_s$ decreases by the same amount. When $c_s = 0$, the budget is immediately recharged to $Q_s$ and a new server deadline is generated as $d_{s,k+1} = d_{s,k} + T_s$.

The CBS is called *active* at time $t$ if pending jobs exist (since $c_s > 0$ always holds), and *idle* otherwise.

## IV. CBS Algorithm

The CBS behavior is governed by three rules, formalized in Buttazzo [1], p. 192 (Figure 6.15).

**Rule 1 — New job arrival when server is idle, $r_j + (c_s/Q_s) \cdot T_s \geq d_{s,k}$:** A new server period is started. Set $d_{s,k+1} = r_j + T_s$ and $c_s \leftarrow Q_s$. Assign deadline $d_{s,k+1}$ to the job.

**Rule 2 — New job arrival when server is idle, $r_j + (c_s/Q_s) \cdot T_s < d_{s,k}$:** The current budget is still fresh. Assign the existing deadline $d_{s,k}$ to the job without replenishment. The budget $c_s$ remains unchanged.

**Rule 3 — Budget exhaustion ($c_s = 0$):** Generate a new deadline $d_{s,k+1} = d_{s,k} + T_s$ and replenish $c_s \leftarrow Q_s$. The running job continues with the new (later) deadline, effectively reducing its EDF priority.

When the server is active (already serving a job), arriving jobs are enqueued in FIFO order. When a job finishes, the next pending job is served using the current budget and deadline.

The condition in Rule 1 vs. Rule 2 determines whether the remaining budget is "fresh enough" to serve the new job within the current server period without causing the demand to exceed $U_s$. This condition is the key mechanism that distinguishes CBS from TBS and prevents the overload interference that affects DS.

## V. Scheduling Example

Following the example in Buttazzo [1], pp. 190–191 (Figure 6.14), consider a hard periodic task $\tau_1$ with $C_1 = 4$, $T_1 = 7$, and a soft task $\tau_2$ served by a CBS with $Q_s = 3$, $T_s = 8$ (so $U_s = 3/8$). The schedulability condition gives: $4/7 + 3/8 = 0.946 \leq 1$ — the system is feasible.

The first job of $\tau_2$ arrives at $r_1 = 3$ when the server is idle. Since $c_s \geq (d_0 - r_1) \cdot U_s$, Rule 1 applies: a new deadline $d_1 = r_1 + T_s = 11$ is set and $c_s = 3$. At $t = 7$, the budget is exhausted (Rule 3): $d_2 = 11 + 8 = 19$ and $c_s = 3$. Since $d_2 = 19 > d_{\tau_1} = 14$, $\tau_1$ preempts and executes. $\tau_2$ resumes at $t = 11$ and finishes at $t = 12$ with $c_s = 2$.

The second job arrives at $r_2 = 13$. Since $c_s = 2 < (d_2 - r_2) \cdot U_s$, Rule 2 applies: the existing deadline $d_2 = 19$ is reused. At $t = 15$, budget exhaustion again triggers Rule 3: $d_3 = 27$, $c_s = 3$. $\tau_1$ preempts until $t = 19$, after which $\tau_2$ finishes the second job at $t = 20$.

Throughout this example, $\tau_1$ meets all its deadlines despite $\tau_2$ executing more than $Q_s$ in total — confirming the isolation property.

## VI. CBS Properties

### A. Isolation Theorem

The central property of CBS is formally stated in Buttazzo [1], p. 192, as:

**Theorem 6.8** [1]: *The CPU utilization of a CBS with parameters $(Q_s, T_s)$ is $U_s = Q_s/T_s$, independently of the computation times and arrival pattern of the served jobs.*

This theorem means the CBS behaves like a periodic task with utilization $U_s$ in EDF analysis, regardless of how much work aperiodic tasks actually demand. The proof is given in Abeni and Buttazzo [2].

### B. Feasibility Lemma

**Lemma 6.6** [1]: *Given $n$ periodic hard tasks with total utilization $U_p$ and $m$ CBS servers with total utilization $U_s = \sum_{i=1}^{m} U_{s_i}$, the whole set is schedulable by EDF if and only if:*

$$U_p + U_s \leq 1 \tag{2}$$

This result allows CBS to be integrated into standard EDF schedulability analysis without any additional complexity — a significant advantage over DS.

### C. EDF Equivalence Lemma

**Lemma 6.7** [1]: *A hard task $\tau_i$ with parameters $(C_i, T_i)$ is schedulable by a CBS with $Q_s \geq C_i$ and $T_s = T_i$ if and only if $\tau_i$ is schedulable with EDF.*

This lemma shows that CBS degenerates to a plain EDF algorithm when serving tasks whose parameters match the server parameters exactly.

### D. Worst-Case Response Time

Buttazzo [1], pp. 196–198, derives the worst-case response time for a job with computation time $C_i$ served by a CBS:

$$R_i = C_i + \left\lceil \frac{C_i}{Q_s} \right\rceil (T_s - Q_s) \tag{3}$$

This answers the open question identified in Milestone 1. The minimum worst-case response time $C_i/U_s$ is achieved when $C_i$ is a perfect multiple of $Q_s$. When context-switch overhead $\epsilon$ is included, the response time becomes:

$$R_i = C_i + \left\lceil \frac{C_i}{T_s U_s - \epsilon} \right\rceil (T_s - T_s U_s + \epsilon) \tag{4}$$

The optimal $T_s$ minimizing average response time is given by Buttazzo [1], p. 201, Eq. (6.15):

$$T_s = \frac{1}{U_s} \left( \epsilon + \sqrt{\frac{\epsilon C^{\text{avg}}}{1 - U_s}} \right) \tag{5}$$

## VII. Critical Evaluation

### A. Strengths

CBS provides several concrete technical advantages:

- **Formal isolation guarantee:** Theorem 6.8 ensures that aperiodic overruns cannot affect periodic task schedulability, unlike DS whose budget preservation can violate the RM bound.
- **Work-conserving:** CBS exploits available slack immediately when a job arrives and the budget is fresh (Rule 2), providing better aperiodic response time than non-work-conserving approaches such as PS.
- **Simple schedulability test:** Eq. (2) requires no separate analysis beyond the standard EDF test.
- **Closed-form response time bound:** Eq. (3) provides a tractable worst-case analysis.
- **Low runtime overhead:** $O(1)$ server state update per scheduling event; constant memory per server instance.

### B. Limitations and Hidden Assumptions

CBS has several significant practical constraints:

- **EDF dependency:** CBS requires an EDF-capable scheduler. Many industrial RTOS (AUTOSAR OS, classic FreeRTOS) use fixed-priority scheduling and cannot run CBS without modification. The Sporadic Server is the appropriate fixed-priority analog.
- **No hard per-job guarantee:** CBS bounds bandwidth, not individual aperiodic job deadlines. Hard aperiodic tasks require different mechanisms.
- **Static parameter selection:** $Q_s$ and $T_s$ are chosen offline. Under variable aperiodic load, a fixed pair may be either wasteful or insufficiently isolating.
- **Single-processor formulation:** The original CBS does not extend directly to multi-core platforms [1].
- **Implicit deadline assumption:** Lemma 6.6 assumes $D_i = T_i$ for periodic tasks. Constrained deadlines require further analysis.

### C. Comparison with TBS

Simulation results in Buttazzo [1], pp. 194–196 (Figures 6.16–6.18), show that CBS and TBS achieve comparable mean tardiness under moderate soft load. However, CBS outperforms TBS when task computation times have high variance (Figure 6.18 in [1]), because TBS relies on exact WCET knowledge at job arrival, which may be unavailable or unreliable. The CBS uses an explicit budget to decouple server behavior from actual execution times, which is the key engineering advantage over TBS in systems with unpredictable aperiodic workloads.

## VIII. Application Analysis

**Suitable scenario — Robotic arm controller:** A system with periodic joint control at 1 kHz (hard RT, $U_p = 0.6$) and soft event-driven network handlers. A CBS with $U_s = 0.3$ reserves 30% bandwidth for handlers. By Lemma 6.6, $0.6 + 0.3 = 0.9 \leq 1$ — schedulable. Network overruns cannot affect the control loop. Linux `SCHED_DEADLINE` implements a CBS-like mechanism suitable for this scenario.

**Suitable scenario — Multimedia streaming:** Buttazzo [1] identifies continuous media applications as the primary motivation for CBS. Variable-length codec outputs create unpredictable execution times — exactly the condition where CBS outperforms TBS (Figure 6.18 in [1]).

**Unsuitable scenario — AUTOSAR automotive ECU:** AUTOSAR OS mandates fixed-priority scheduling (OSEK standard). CBS cannot be deployed without replacing the scheduler. The Sporadic Server is the correct mechanism in this context.

**Unsuitable scenario — Hard aperiodic tasks:** CBS provides bandwidth guarantees, not per-job deadline guarantees. Safety-critical interrupt handlers requiring certified response time bounds cannot rely on CBS alone.

## IX. Conclusion

The Constant Bandwidth Server is a formally rigorous and practically useful mechanism for integrating aperiodic task service into EDF-scheduled real-time systems. Its isolation property (Theorem 6.8) and simple feasibility test (Lemma 6.6) make it analytically superior to its predecessors, particularly the Deferrable Server. The closed-form worst-case response time (Eq. 3) and optimal parameter selection formula (Eq. 5) provide tractable design tools.

The primary limitation is the requirement for an EDF runtime, which restricts deployment on fixed-priority RTOS. For systems that can support EDF, CBS provides a principled solution to the coexistence of hard periodic and soft aperiodic workloads, with performance comparable to TBS under normal conditions and superior isolation under overload.

Open directions include multi-core CBS extensions, adaptive parameter tuning, and integration with resource-sharing protocols such as SRP.

## Acknowledgment

AI assistance was used during milestone preparation and is documented in the AI usage protocols submitted alongside this paper (Milestones 1–3). All technical claims in this paper were verified against primary sources [1]–[3].

## References

[1] G. C. Buttazzo, *Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications*, 4th ed. Springer, 2024, Ch. 6, pp. 189–203.

[2] L. Abeni and G. Buttazzo, "Integrating Multimedia Applications in Hard Real-Time Systems," in *Proc. 19th IEEE Real-Time Systems Symposium (Cat. No. 98CB36279)*, IEEE, 1998, pp. 4–13.

[3] L. Abeni and G. Buttazzo, "Resource Reservation in Dynamic Real-Time Systems," *Real-Time Systems*, vol. 27, no. 2, pp. 123–167, 2004.

[4] S. Ferdous, *AI Usage Protocols, Milestones 1–3: Constant Bandwidth Server*, HSHL, 2026. [Protocol v0.2; tools: ChatGPT GPT-5.5 (M1), Claude claude-sonnet-4-6 (M2, M3); JSON and .bib files submitted per milestone.]